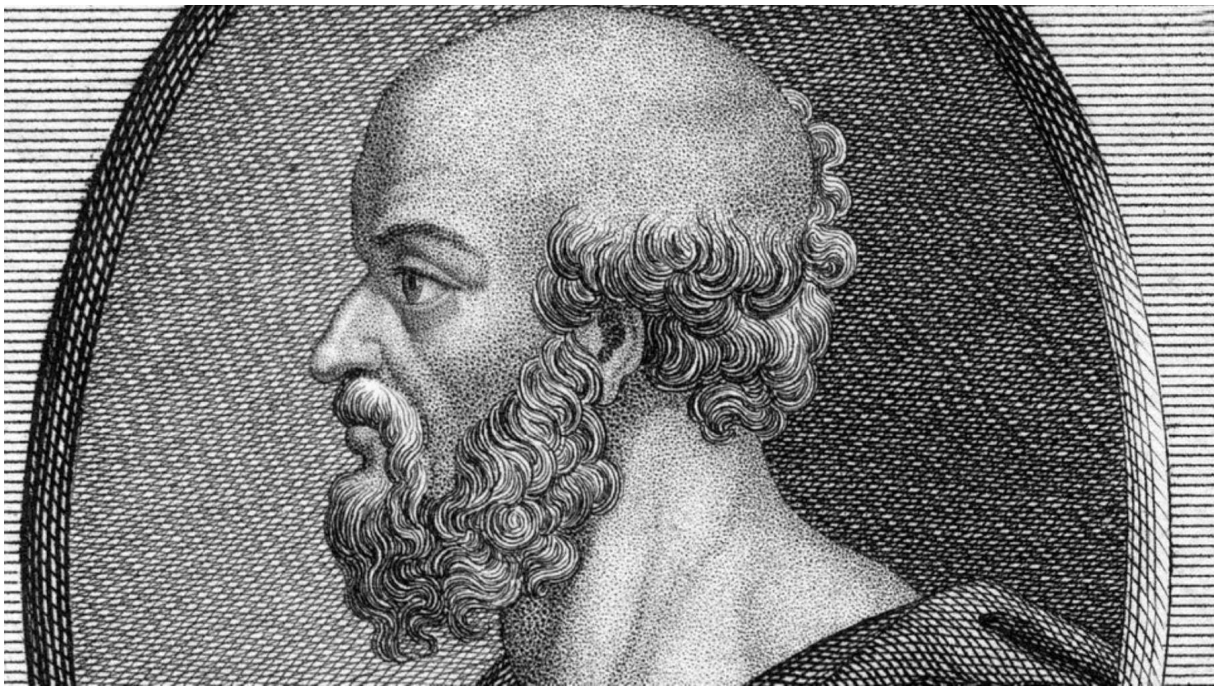
ETSIIT
Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

Ingeniería Informática + ADE

Universidad de Granada (UGR)

Autor: Ismael Sallami Moreno

Asignatura: Sistemas Concurrentes y Distribuidos: Criba de Eratóstenes



Índice

1. Enunciado de la Actividad **3**

2. Solución con MPI **3**

2.1. Explicación detallada del código: Algoritmo de la Criba de Eratóstenes con MPI . . . 5

2.1.1. Descripción del Algoritmo . . . 5

2.1.2. Implementación . . . 5

2.1.3. Detalles de Implementación . . . 6

2.2. Compilación . . . 6

2.3. Ejecución y/o salida . . . 6

2.4. Demostración Matemática de la Correctitud del Algoritmo . . . 7

2.4.1. Definición de Número Primo . . . 7

2.4.2. Esquema del Algoritmo . . . 7

2.4.3. Correctitud del Algoritmo . . . 8

2.4.4. Conclusión del Análisis . . . 8

1 Enunciado de la Actividad

El objetivo de este ejercicio es programar una versión distribuida del famoso método de *la criba de Erastotenes* para obtener los N primeros números primos. Se utilizará una red de procesos concurrentes conectados por una tubería o "pipeline"de procesos, como en la figura siguiente, para generar los N primeros números primos.

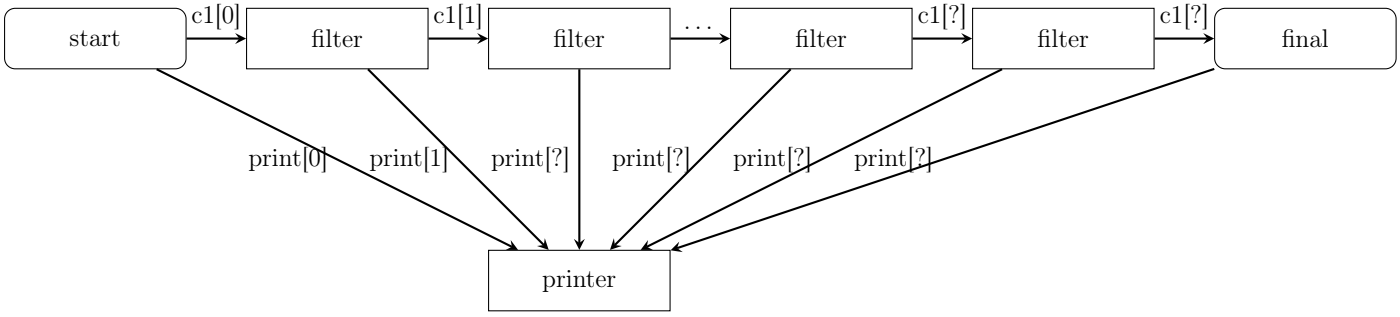


Figura 1: Pipeline de procesos para generar números primos

Para programarlo, suponer que se conoce una constante `nLimite` que es mayor o igual que el N-ésimo número primo. El proceso `start` generará el primer número primo (el 2) y lo enviará al proceso `printer`, que lo imprime; a continuación genera el siguiente primo (el 3), lo envía al primer proceso `filter`, al que seguirá enviando los siguientes números impares: 5, 7, 9, y así sucesivamente.

2 Solución con MPI

La solución propuesta consiste en ejecutar el programa de la criba de Eratóstenes utilizando MPI. A medida que se incrementa el número de procesos especificados con ‘mpirun‘, se añaden más filtros, lo que mejora la precisión y eficiencia del resultado.

```
#include <mpi.h>
#include <iostream>
using namespace std;

int main(int argc, char *argv[]) {
    int rank, size, x, valor;        // Variables para identificar rango, tamaño y valores
    bool fin = false;               // Indica si el procesamiento debe finalizar
    MPI_Status status;              // Estado para recibir información de mensajes
    const int nLimite = 100;        // Número límite para el cálculo de números primos

    MPI_Init(&argc, &argv);                        // Inicializa el entorno MPI
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);          // Obtiene el rango del proceso actual
    MPI_Comm_size(MPI_COMM_WORLD, &size);          // Obtiene el número total de procesos
```

```

    if (size < 3) {
        if (rank == 0) {
            cerr << "Se requieren al menos 3 procesos para ejecutar este programa." << endl;
        }
        MPI_Abort(MPI_COMM_WORLD, 1); // Aborta la ejecución si hay menos de 3 procesos
    }

    if (rank == 0) {
        // Proceso principal: genera números y los envía al primer filtro
        for (x = 2; !fin; x++) { // Itera sobre números naturales comenzando desde 2
            MPI_Send(&x, 1, MPI_INT, 1, 0, MPI_COMM_WORLD); // Envía cada número al primer filtro
            if (x > nLimite) { // Determinamos como rango máximo el nLimite
                int signal = -1; // Señal de finalización del proceso
                MPI_Send(&signal, 1, MPI_INT, 1, 0, MPI_COMM_WORLD); // Envía la señal de finalización
                fin = true; // Finaliza el bucle
            }
        }
    } else if (rank == size - 1) {
        // Último proceso (impresor): recibe números primos y los imprime
        if (rank == size - 1) {
            cout << "Proceso " << rank << ": 2 es primo." << endl; // Imprime que 2 es primo
            cout << "Proceso " << rank << ": 3 es primo." << endl; // Imprime que 3 es primo
        }

        while (!fin) {
            MPI_Recv(&valor, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD, &status); // Recibe números del filtro anterior
            if (valor == -1) { // Señal de finalización
                fin = true; // Finaliza el bucle
            } else {
                cout << "Proceso " << rank << ": " << valor << " es primo." << endl; // Imprime el número primo
            }
        }
    } else {
        // Procesos intermedios (filtros): filtran números según su rango
        MPI_Recv(&valor, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD, &status); // Recibe el primer número primo
        int primo = valor; // Almacena el número primo que filtra
        while (!fin) {
            MPI_Recv(&valor, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD, &status); // Recibe el siguiente número
            if (valor == -1) { // Señal de finalización
                MPI_Send(&valor, 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD); // Propaga la señal de finalización
                fin = true; // Finaliza el bucle
            } else if (valor % primo != 0) { // Si el número no es divisible por el primo actual
```

```
                if (rank < size - 2) {
                    MPI_Send(&valor, 1, MPI_INT, rank + 1, 0,
                        MPI_COMM_WORLD); // Envía al siguiente filtro
                } else {
                    MPI_Send(&valor, 1, MPI_INT, size - 1, 0,
                        MPI_COMM_WORLD); // Envía al impresor
                }
            }
        }
    }

    MPI_Finalize(); // Finaliza el entorno MPI
    return 0;
}
```

2.1. Explicación detallada del código: Algoritmo de la Criba de Eratóstenes con MPI

En esta implementación, utilizamos el modelo de programación paralelo de *Message Passing Interface* (MPI) para distribuir el trabajo de la Criba de Eratóstenes entre varios procesos. La idea principal es dividir la tarea de identificar números primos en un rango determinado, permitiendo que cada proceso se encargue de una parte específica del cálculo. Siguiendo el esquema de la Figura 1.

2.1.1. Descripción del Algoritmo

El algoritmo se organiza en tres roles principales para los procesos:

- **Proceso principal** (`rank == 0`): Este proceso es responsable de generar números naturales, comenzando desde el número 2, y enviarlos al primer proceso filtro (`rank == 1`). Los números se envían uno a uno utilizando `MPI_Send`. Además, este proceso envía una señal de finalización (`-1`) cuando se ha alcanzado el rango máximo (en este caso, 100).

- **Procesos filtros intermedios** (`1 <= rank <size - 1`): Cada proceso filtro recibe números del proceso anterior y aplica un filtro basado en la divisibilidad por el número primo asociado a ese filtro. Los números no divisibles se envían al siguiente filtro, y el proceso finaliza cuando recibe la señal de terminación.

- **Proceso impresor** (`rank == size - 1`): Este proceso imprime los números primos recibidos. También maneja explícitamente los casos especiales del número 2 y 3, imprimiendo directamente que son primos al inicio del programa.

2.1.2. Implementación

La implementación comienza inicializando el entorno MPI con `MPI_Init`. Luego, cada proceso determina su rol basándose en su rango (`rank`):

- En el **proceso principal**, un bucle itera sobre números naturales, comenzando en 2. Cada número se envía al primer filtro mediante `MPI_Send`. Cuando se supera el

rango de números (100 en este caso), el proceso envía una señal de terminación (-1) y finaliza.

- Los **procesos filtros intermedios** comienzan recibiendo un número primo inicial, que se convierte en su referencia para filtrar. Posteriormente, filtran los números recibidos y envían los que no son divisibles al siguiente filtro o al proceso impresor. Cuando reciben la señal de terminación (-1), la propagan al siguiente proceso y finalizan.

- El **proceso impresor** recibe números desde el último filtro y los imprime como primos. Adicionalmente, este proceso imprime explícitamente los números 2 y 3 como primos al inicio, asegurando que no sean ignorados por los filtros.

Finalmente, el entorno MPI se finaliza utilizando `MPI_Finalize`.

2.1.3. Detalles de Implementación

Para garantizar que los números 2 y 3 sean manejados adecuadamente:

- El proceso impresor imprime explícitamente estos valores al inicio.

- En el proceso principal, los números 2 y 3 se envían al primer filtro y también directamente al impresor.

Esta estructura asegura una distribución eficiente del trabajo entre los procesos, con una adecuada coordinación entre ellos para identificar y reportar los números primos dentro del rango especificado.

2.2. Compilación

```
# Compilar el código :
    mpic++ -o criba criba.cpp
# Ejecutar el programa con 4 procesos:
    mpirun -np 4 ./criba
```

2.3. Ejecución y/o salida

```
Proceso 3: 2 es primo.
Proceso 3: 3 es primo.
Proceso 3: 5 es primo.
Proceso 3: 7 es primo.
Proceso 3: 11 es primo.
Proceso 3: 13 es primo.
Proceso 3: 17 es primo.
Proceso 3: 19 es primo.
Proceso 3: 23 es primo.
Proceso 3: 25 es primo.
Proceso 3: 29 es primo.
Proceso 3: 31 es primo.
Proceso 3: 35 es primo.
```

```
Proceso 3: 37 es primo.
Proceso 3: 41 es primo.
Proceso 3: 43 es primo.
Proceso 3: 47 es primo.
Proceso 3: 49 es primo.
Proceso 3: 53 es primo.
Proceso 3: 55 es primo.
Proceso 3: 59 es primo.
Proceso 3: 61 es primo.
Proceso 3: 65 es primo.
Proceso 3: 67 es primo.
Proceso 3: 71 es primo.
Proceso 3: 73 es primo.
Proceso 3: 77 es primo.
Proceso 3: 79 es primo.
Proceso 3: 83 es primo.
Proceso 3: 85 es primo.
Proceso 3: 89 es primo.
Proceso 3: 91 es primo.
Proceso 3: 95 es primo.
Proceso 3: 97 es primo.
Proceso 3: 101 es primo.
```

2.4. Demostración Matemática de la Correctitud del Algoritmo

El algoritmo implementado se basa en la Criba de Eratóstenes, un método clásico para encontrar números primos en un rango dado. A continuación, demostramos matemáticamente que este enfoque es correcto.

2.4.1. Definición de Número Primo

Un número $n > 1$ es primo si y solo si no es divisible por ningún número entero k tal que $2 \leq k \leq \sqrt{n}$. Es decir, no existen divisores propios de n en el rango mencionado.

2.4.2. Esquema del Algoritmo

El algoritmo utiliza un conjunto de procesos para filtrar números no primos de manera iterativa:

1. El primer proceso (`rank == 0`) genera números consecutivos x comenzando desde 2 y los envía al primer filtro.

2. Cada proceso filtro P_i conserva un número primo p_i, recibido del proceso anterior. Filtra los números x recibidos eliminando aquellos que satisfacen $x \mod p_i = 0$ (es decir, los múltiplos de p_i).

3. Los números que no son múltiplos de p_i son enviados al siguiente filtro.

4. Finalmente, el último proceso (`rank == size - 1`) imprime todos los números que llegan hasta él, ya que no han sido descartados por ningún filtro previo.

2.4.3. Correctitud del Algoritmo

Para demostrar que este algoritmo es correcto, debemos probar que:

1. **Todo número primo es identificado correctamente.**
2. **Todo número compuesto es eliminado correctamente.**

Identificación de números primos Por definición, un número primo p no tiene divisores propios en el rango $[2, \sqrt{p}]$. En el algoritmo:

- El primer filtro retiene 2, el siguiente retiene 3, y así sucesivamente.
- Un número p que llega al filtro P_i no es divisible por $p_1, p_2, \ldots, p_{i-1}$, ya que estos filtros han descartado previamente los múltiplos de sus primos asociados.
- Si p llega al último filtro, entonces p no tiene divisores en $[2, \sqrt{p}]$, lo que implica que es primo.

Por lo tanto, el algoritmo identifica todos los números primos correctamente.

Eliminación de números compuestos Sea n un número compuesto. Por definición, n tiene al menos un divisor d tal que $2 \leq d \leq \sqrt{n}$. En el algoritmo:

- El filtro correspondiente al primo d descarta n, ya que $n \mod d = 0$.
- Como $d \leq \sqrt{n}$, el filtro asociado a d se encuentra antes de los procesos que manejan n, asegurando su eliminación.

Por lo tanto, ningún número compuesto llega al último proceso impresor.

2.4.4. Conclusión del Análisis

El algoritmo implementado en MPI garantiza que:

- Todos los números primos dentro del rango son impresos.
- Ningún número compuesto es identificado como primo.

Esto demuestra matemáticamente que el algoritmo es correcto y cumple con los requisitos de la Criba de Eratóstenes para la identificación de números primos.